SOFTWARE ENGINEERING LAB | LAB 25

# Rate Limiting & Backpressure di Go

Arsitektur Token Bucket, Bounded Queue Backpressure, RFC 6585 HTTP 429, dan AWS Full Jitter Retry

**Lukman / lukman-ss**

# Problem: Beban Ekstrem & Antrean Tak Terbatas

Ketiadaan kontrol beban memicu penumpukan antrean memori dan kegagalan kaskade.

### MOTIVASI

Sistem terdistribusi menghadapi ketidakseimbangan struktural ketika laju kedatangan request masuk melampaui kemampuan downstream worker. Tanpa mekanisme pembatasan, request menumpuk di memori tanpa batas, memicu degradasi latensi dramatis, OOM crashes, dan kegagalan total sistem.

### BATAS LAB

Lab ini mengimplementasikan Token Bucket, Leaky Bucket, bounded queue channel dengan fast rejection, middleware HTTP 429 RFC 6585, dan exponential backoff AWS Full Jitter menggunakan pustaka standar Go 1.22+ murni.

**Rate limiting protects ingress boundaries; backpressure protects internal execution pipelines.**

# Mental Model: Batas Sistem vs. Aliran Internal

Membedakan peran penjaga batas gerbang dan katup tekanan internal.

### PENJAGA GERBANG (RATE LIMITER)

Rate limiter beroperasi di boundary sistem (API Gateway / middleware). Menentukan apakah klien tertentu diizinkan masuk berdasarkan kuota waktu. Jika kuota habis, tolak seketika dengan respon HTTP 429 Too Many Requests dan header Retry-After.

### KATUP INTERNAL (BACKPRESSURE)

Backpressure beroperasi di internal pipeline pemrosesan. Mengalirkan sinyal tekanan mundur saat worker downstream kelebihan beban. Bounded queue menolak penambahan antrean baru (fast rejection) demi menjaga kesehatan worker yang sedang bekerja.

**Rate limiting controls admission quota; backpressure prevents internal worker saturation.**

# Arsitektur Pertahanan 4 Lapis

Alur end-to-end penanganan request dari ingress hingga retry klien.

## ALUR INGRESS & LIMITER

1. Request Masuk diterima HTTP Middleware.
2. Ekstraksi Tenant Key (X-API-Key) mencegah tabrakan CGNAT.
3. Evaluasi Token Bucket: jika kuota habis, respon HTTP 429 seketika.
4. Request yang lolos diteruskan ke layer antrean.

## ALUR QUEUE & RETRY

5. Bounded Queue menerima job via select non-blocking.
6. Bila buffer penuh, tolak cepat dengan ErrQueueFull.
7. Worker Pool memproses job stabil sesuai Hukum Little.
8. Klien menerapkan retry AWS Full Jitter untuk meredam badai.

**Defense-in-depth: boundary throttling, bounded buffering, and desynchronized client retries.**

# Token Bucket vs. Leaky Bucket

Perbandingan mendalam dua algoritma fundamental pengendalian laju.

### TOKEN BUCKET (BURST TOLERANT)

- Karakteristik: Mengizinkan lonjakan (burst) hingga kapasitas B.
- Refill: Token terisi secara kontinu pada laju R.
- Konsumsi: Setiap request mengambil 1 token jika tersedia.
- Kasus Ideal: REST API publik, interaksi user, Stripe gateway.

### LEAKY BUCKET (TRAFFIC SMOOTHING)

- Karakteristik: Menegakkan laju pengeluaran konstan R.
- Pengisian: Request menambah air ke wadah berkapasitas B.
- Kebocoran: Air menetes pada laju tetap R; tolak jika meluap.
- Kasus Ideal: Egress network packet pacing, NGINX limit_req.

**Token bucket permits controlled bursts; leaky bucket enforces strict constant output pacing.**

# Formulasi Matematis Token Bucket

Kalkulasi pengisian token berbasis waktu monotonik tanpa timer latar.

## RUMUS PENGISIAN TOKEN

Perhitungan token baru saat request tiba pada waktu t:
elapsed = now.Sub(lastRefill).Seconds()
tokens = min(capacity, currentTokens + elapsed * refillRate)

Jika tokens >= n, kurangi tokens -= n dan izinkan request.

## KALKULASI RETRY-AFTER

Ketika token tidak mencukupi, durasi tunggu dihitung deterministik:
needed = requestedTokens - currentTokens
retryAfter = ceil(needed / refillRate)

Nilai integer detik ini dikirimkan via header HTTP standard RFC 6585.

**Monotonic timestamp delta calculation yields zero-background-thread token replenishment.**

# Implementasi Token Bucket di Go

Struktur data thread-safe dengan sinkronisasi sync.Mutex.

### STRUKTUR DATA MUTEX

```
type TokenBucket struct {
mu sync.Mutex
capacity float64
tokens float64
refillRate float64
lastRefill time.Time
}
```

Floating-point digunakan untuk presisi akumulasi token fraksional.

### LOGIKA IZIN ATOMIK

```
func (tb *TokenBucket) AllowN(now time.Time, n float64) bool {
tb.mu.Lock()
defer tb.mu.Unlock()
// update tokens via elapsed time
if tb.tokens >= n {
tb.tokens -= n; return true
}
return false
}
```

**Atomic mutex-protected state updates prevent race conditions across parallel requests.**

# Implementasi Leaky Bucket di Go

Pemodelan tetesan konstan dan pencegahan overflow antrean.

### STRUKTUR DATA LEAKY

```
type LeakyBucket struct {
mu sync.Mutex
capacity float64
waterLevel float64
leakRate float64
lastLeak time.Time
}
```

Air (waterLevel) merepresentasikan akumulasi request tertunda.

### LOGIKA PENIRISAN AIR

```
func (lb *LeakyBucket) Allow() bool {
lb.mu.Lock()
defer lb.mu.Unlock()
elapsed := time.Since(lb.lastLeak).Seconds()
lb.waterLevel = math.Max(0, lb.waterLevel - elapsed*lb.leakRate)
if lb.waterLevel + 1.0 <= lb.capacity {
lb.waterLevel += 1.0; return true
}
return false
}
```

**Leaky bucket drains water continuously at fixed rate, dropping excess when full.**

internal/ratelimit/bucket.go:78-132

# Registry & Isolasi Tenant RFC 6598

Mencegah fenomena noisy-neighbor dan tabrakan kuota Carrier-Grade NAT.

### BAHAYA RATE LIMIT BY IP

Di bawah Carrier-Grade NAT (RFC 6598), ribuan pengguna seluler berbagi 1 IP publik (100.64.0.0/10). Pembatasan berbasis IP akan memblokir pengguna normal hanya karena satu pengguna lain menghabiskan kuota.

### REGISTRY BERBASIS API-KEY

Registry memetakan API Key atau Tenant ID unik ke instance TokenBucket masing-masing via sync.RWMutex. Klien anonymous diarahkan ke bucket bersama terpisah, memastikan kuota terisolasi sempurna.

**Per-tenant registry keying isolates noisy neighbors and avoids Carrier-Grade NAT collisions.**

# Bounded Queue & Non-Blocking Submit

Menolak beban berlebih seketika tanpa mengorbankan memori heap.

### BAHAYA UNBOUNDED BUFFER

Antrean tanpa batas menyembunyikan masalah kelebihan beban. Request yang menunggu terlalu lama akan kadaluarsa di sisi klien (client timeout), namun worker server tetap memprosesnya secara sia-sia.

### SELECT-DEFAULT ZERO-BLOCK

```
select {
case bq.jobs <- job:
bq.stats.Accepted++
return nil
default:
bq.stats.Rejected++
return ErrQueueFull
}
```

Penolakan terjadi dalam skala sub-mikrodetik tanpa alokasi memori heap.

**Non-blocking channel selection yields instant rejection on queue saturation.**

# Hukum Little (L = λW) dalam Antrean

Fondasi matematis analisis kapasitas antrean dan estimasi penundaan.

## FORMULASI HUKUM LITTLE

L = λ * W

- L = Jumlah rata-rata item dalam sistem antrean.
- λ = Laju kedatangan request per detik (arrival rate).
- W = Rata-rata waktu tunggu item dalam antrean.

Jika λ > laju proses μ, W membesar proporsional terhadap waktu t.

## SKENARIO DRAIN TIME

Jika antrean menampung 5.000.000 request dan kapasitas proses worker adalah 2.000 req/detik:

Waktu Habis = 5.000.000 / 2.000 = 2.500 detik (~41 menit 40 detik).

Klien sudah lama memutus koneksi sebelum request selesai diproses.

**Little's Law proves unbounded queues accumulate catastrophic processing delay.**

# Worker Pool & Graceful Shutdown

Pembersihan aman, pengosongan buffer, dan pencegahan job bocor.

### EKSEKUSI WORKER TANK

Worker membaca channel berkapasitas tetap hingga menerima sinyal context.Done() atau penutupan channel. Setiap worker menangani job secara sekuensial dan memperbarui metrik statistik Processed secara atomik.

### METODE STOP() AMAN

Metode Stop() mengunci mutex, menandai stopped=true, membatalkan context, menutup channel jobs, dan menunggu seluruh goroutine selesai via sync.WaitGroup. Tidak ada goroutine yang menggantung (leak).

**Deterministic graceful shutdown drains in-flight jobs without goroutine leakage.**

# HTTP 429 RFC 6585 & Retry-After

Standar respon kelebihan beban HTTP dan pencegahan caching perantara.

## STATUS CODE RFC 6585

HTTP 429 Too Many Requests menandakan klien melampaui kuota laju yang ditentukan.

- Header Wajib: Retry-After:
- Payload JSON: {"error": "rate_limit_exceeded", "retry_after": N}
- Larangan Cache: Respon 429 TIDAK BOLEH di-cache oleh CDN/Proxy.

## PERBEDAAN 429 VS 503

- 429 Too Many Requests: Penolakan spesifik kuota klien (client quota exceeded).
- 503 Service Unavailable: Kegagalan kapasitas internal server (server overloaded).

Memberikan kode status yang tepat membantu klien menentukan strategi retry.

**RFC 6585 status 429 signals client quota violation with deterministic Retry-After header.**

# Thundering Herd & Bahaya Retry Sinkron

Bagaimana pengulangan request deterministik melumpuhkan proses pemulihan.

### SINKRONISASI FASA

Jika 100 klien gagal pada waktu t=0 dan mengulang dengan interval eksponensial tetap (misal t=1s, 2s, 4s), seluruh 100 klien akan memukul server secara serempak di setiap detik yang sama. Puncak beban berlipat ganda.

### DAMPAK PADA SERVER

Lonjakan beban periodik ini mencegah downstream worker pulih dari kegagalan awal. Sistem terperangkap dalam lingkaran setan (retry storm): setiap gelombang pengulangan memicu penolakan baru dan badai lanjutan.

**Deterministic retries synchronize into thundering herds that crush recovering services.**

# Formulasi AWS Full Jitter

Penyebaran acak seragam untuk melenyapkan korelasi waktu pengulangan.

## RUMUS MARC BROOKER (AWS)

sleep = random(0, min(cap, base * 2^attempt))

- base: Jeda awal (misal 50ms)
- cap: Batas jeda maksimum (misal 10s)
- attempt: Indeks percobaan pengulangan (0, 1, 2...)

Nilai jeda disebar acak dari 0 hingga batas eksponensial.

## KOMPARASI DENGAN VARIAN LAIN

- Equal Jitter: sleep = half + random(0, half) — mempertahankan jeda minimal.
- Decorrelated Jitter: sleep = min(cap, random(base, prevSleep * 3)).

Penelitian AWS membuktikan Full Jitter meminimalkan total waktu kerja server.

**Full Jitter uniformly distributes retry timestamps, eliminating synchronized thundering herds.**

# Implementasi Retry Backoff di Go

Modul evaluasi strategi backoff modular dan batas numerik aman.

## FUNGSI BACKOFF DI GO

```
func ComputeBackoff(strat JitterStrategy, attempt int, base, cap, prev time.Duration)
time.Duration {
maxSleep := float64(base) * math.Pow(2, float64(attempt))
if maxSleep > float64(cap) {
maxSleep = float64(cap)
}
// evaluasi switch case strategi
}
```

## PENJAGA BATAS NUMERIK

Fungsi menjamin hasil perhitungan tidak melampaui cap dan menangani batas overflow matematis pada attempt tinggi. Strategi No Jitter, Full Jitter, Equal Jitter, dan Decorrelated Jitter didukung penuh.

**Strict boundary checks prevent integer overflow during high retry attempt iterations.**

# Eksekusi Demonstrasi: 4 Tahap Terpadu

Verifikasi perilaku Token Bucket, Leaky Bucket, Queue, dan Jitter secara live.

### TAHAP 1 & 2: BUCKET DEMO

• Tahap 1 Token Bucket (Cap 3, Rate 5/s): 3 request pertama sukses, request 4-5 ditolak. Setelah jeda 300ms, token terisi kembali dan request diizinkan.
• Tahap 2 Leaky Bucket (Cap 3, Leak 10/s): Air terakumulasi hingga kapasitas penuh dan menolak burst berlebih.

### TAHAP 3 & 4: QUEUE & RETRY

• Tahap 3 Bounded Queue (Cap 3, 1 Worker): Menerima 3 job, menolak job 4 & 6 seketika dengan ErrQueueFull. Statistik akhir: Accepted=4, Rejected=2.
• Tahap 4 Retry Comparison: Full Jitter menghasilkan latensi lebih rendah dan sebaran interval merata.

**Comprehensive live demo proves isolation, rejection, and backoff across four phases.**

cmd/demo/main.go:20-140

# Verifikasi Bebas Race: 50 Goroutine Paralel

Pengujian konkurensi ketat menggunakan Go Race Detector (-race).

### SKENARIO UJI STRES

TestTokenBucket_ConcurrencyRace mengeksekusi 50 goroutine secara paralel. Masing-masing goroutine melakukan 100 pemanggilan Allow() secara serempak ke satu instance bucket yang sama (total 5.000 konkurensi).

### INTEGRITAS DATA ATOMIK

Pendeteksi race detector Go melaporkan 0 data race. Jumlah request yang diterima tidak pernah melampaui kapasitas awal ditambah akumulasi pengisian token selama durasi pengujian. State selalu konsisten.

**Race detector guarantees zero memory corruption under high-concurrency goroutine stress.**

# Matriks Verifikasi: 16/16 Unit Tests Pass

Cakupan pengujian menyeluruh pada seluruh paket internal.

### RATELIMIT & BACKPRESSURE

- TestTokenBucket_BurstAndRefill (PASS)
- TestLeakyBucket_LeakRate (PASS)
- TestRegistry_TenantIsolation (PASS)
- TestBoundedQueue_RejectionUnderLoad (PASS)
- TestBoundedQueue_SubmitAfterStop (PASS)

### HTTPUTIL & RETRY JITTER

- TestRateLimitMiddleware_RFC6585 (PASS)
- TestRateLimitMiddleware_AnonymousFallback (PASS)
- TestComputeBackoff_Bounds (PASS)
- TestDecorrelatedJitter_Bounds (PASS)
- Durasi total suite: ~6.5 detik bebas race.

**Deterministic test suite validates RFC compliance, bounds enforcement, and queue limits.**

# Mekanisme Pemulihan & Graceful Degradation

Strategi pemulihan mandiri sistem saat badai trafik mereda.

### SELF-HEALING OTOMATIS

Token Bucket pulih secara alami tanpa intervensi manual seiring berjalannya waktu. Klien yang mematuhi header Retry-After akan menunda request hingga token terisi kembali, memulihkan stabilitas secara organik.

### STRATEGI LOAD SHEDDING

Bila downstream mengalami kegagalan permanen:

1. Bounded queue menolak beban non-kritis.
2. Tingkatkan visibilitas via queue age monitoring.
3. Turunkan kapasitas bucket secara dinamis untuk memberi ruang bernapas bagi database.

**Self-healing token replenishment restores service health as soon as traffic subsides.**

# Batasan Implementasi Lab vs. Produksi

Kesenjangan arsitektur yang wajib dijembatani pada sistem enterprise multi-node.

### PENYIMPANAN IN-MEMORY

Implementasi lab menyimpan state token di RAM host lokal. Pada cluster multi-server, klien dapat membanjiri instance berbeda. Solusi: Gunakan Redis Cluster dengan skrip Lua atomik atau sorted set sliding window.

### DISTRIBUTED BACKPRESSURE

Channel Go beroperasi dalam satu proses. Backpressure lintas microservices membutuhkan arsitektur message queue terdistribusi (Apache Kafka dengan consumer lag monitoring atau RabbitMQ consumer prefetch).

**Multi-instance production deployments require centralized Redis coordination and Kafka queues.**

# Anti-Patterns & Kesalahan Umum

Jebakan implementasi rate limiting dan backpressure yang sering terjadi.

### KESALAHAN GATEWAY

- Rate limit berbasis IP tanpa memperhitungkan CGNAT (RFC 6598).
- Mengabaikan header Retry-After sehingga klien mengulang membabi buta.
- Membiarkan perantara (CDN/Proxy) menyimpan cache respon HTTP 429.

### KESALAHAN WORKER & RETRY

- Antrean tanpa batas (unbounded channel) yang berakhir pada OOM kill.
- Retry eksponensial tanpa jitter yang memicu thundering herd.
- Memantau kedalaman antrean (depth) alih-alih usia antrean (queue age).

**Avoid IP-only throttling, unbuffered unbounded queues, and jitter-less deterministic retries.**

content/02-master-draft.md:109-116

# Checklist Implementasi Ketahanan Beban

Panduan verifikasi langkah demi langkah untuk arsitek dan insinyur backend.

### DESAIN & RATE LIMITING

[ ] Tentukan boundary proteksi (API Gateway / Ingress).
[ ] Pilih algoritma: Token Bucket (burst) / Leaky (smooth).
[ ] Terapkan isolasi tenant berbasis API-Key (hindari IP murni).
[ ] Implementasikan header Retry-After pada respon 429.

### BACKPRESSURE & RETRY

[ ] Konfigurasikan bounded queue dengan fast rejection.
[ ] Ukur kapasitas antrean berdasarkan Hukum Little (L = λW).
[ ] Terapkan AWS Full Jitter pada klien upstream.
[ ] Validasi konkurensi mutlak dengan go test -race.

**Follow the structured checklist to ensure end-to-end overload resilience across your fleet.**

SOFTWARE ENGINEERING LAB | LAB 25

# Rate Limiting & Backpressure Implementation

Boundary throttling protects service ingress; bounded queue backpressure prevents internal worker collapse; full jitter retry eliminates destructive thundering herds.

- Verifikasi Otomatis: 16/16 PASS (go test -race ./...)
- Kunci Arsitektur: Token Bucket Monotonic Delta, Bounded Queue Fast Rejection, Tenant Isolation RFC 6598, AWS Full Jitter Retry
- Repositori: github.com/lukman-ss/software-engineering-lab/tree/main/labs/25-rate-limiting-and-backpressure
- Komitmen Terverifikasi: a39256cb

**Karya: Lukman / lukman-ss • Software Engineering Lab Seri 25**